Lesson 7: On the Move

Unity

VIDEO GAME CREATION

This week: enemies stop standing still. Your job is to make every enemy advance slowly toward the player, and clean up any that reach the bottom of the screen without being shot.

Open your own [YourName]Game project from Lesson 6 - don't start a new one.

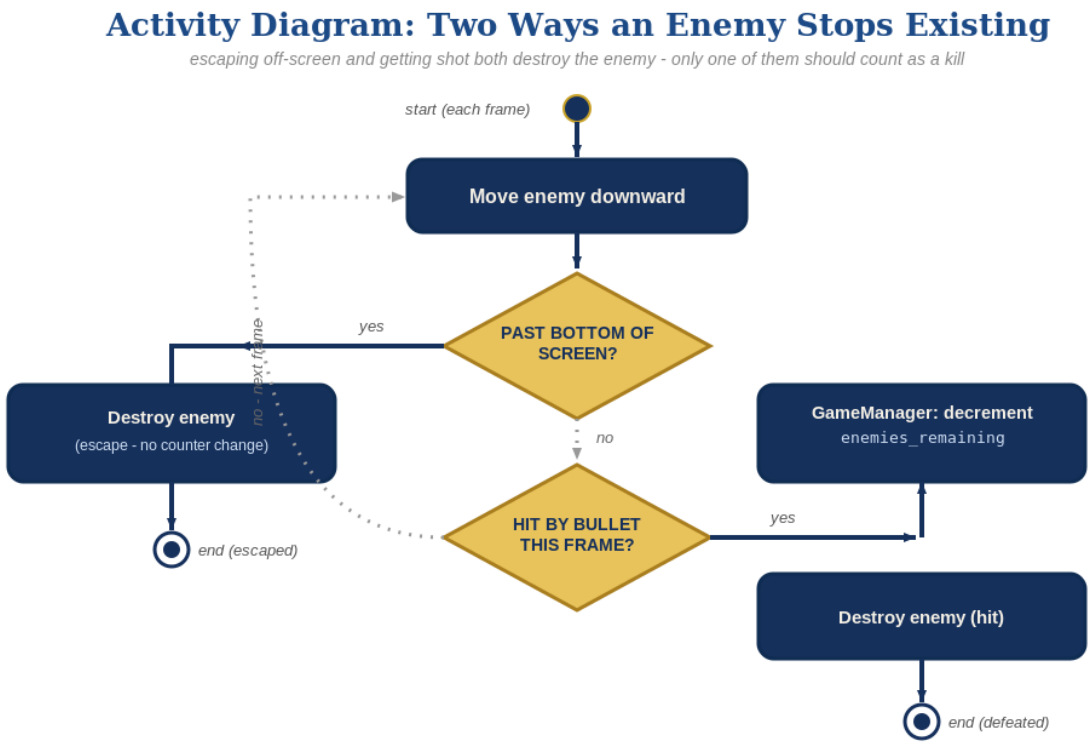
Tip: no new assets this week - this is a logic-only lesson.

Setup

1. Open your [YourName]Game project from Lesson 6.

Level 1 - Minimal (MVP)

Every enemy moves slowly downward, toward the player, instead of sitting still. An enemy that reaches the bottom of the screen unshot is removed - and does NOT count toward your win condition, since it escaped rather than being defeated.



The general escape-vs-hit distinction behind this week's exercise - the same shape in every engine, whatever the exact code or blocks look like.

2. In Enemy.cs, add a public float enemySpeed and an Update() method that moves the enemy downward and destroys it (without calling GameManager.RegisterKill()) if it goes past the bottom of the screen (code below).
3. Press Play and let the enemies drift downward - shooting one should still work exactly as before, and letting one reach the bottom unshot should just destroy it, with no "You Win!" message.

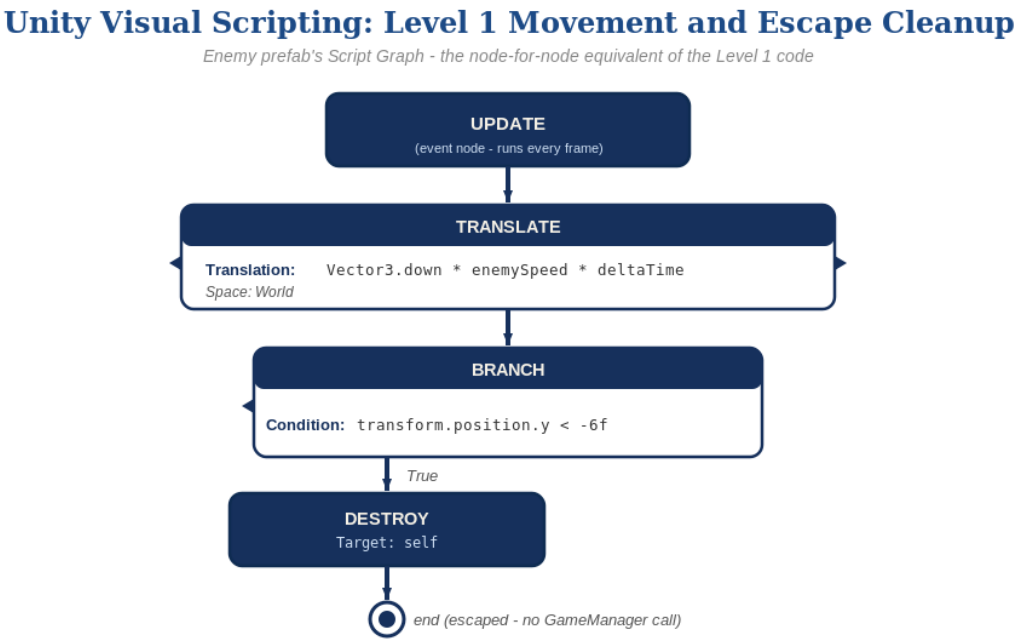
```
// Enemy.cs - add a field and a new method:
public float enemySpeed = 1.5f;

void Update() {
    transform.Translate(Vector3.down * enemySpeed * Time.deltaTime);
    if (transform.position.y < -6f) {
        Destroy(gameObject);
    }
}
```

Alternate Path: Visual Scripting

The same movement-and-cleanup pattern works in a Script Graph instead of C#.

4. Open the Enemy prefab's Script Graph (the same graph carrying its existing On Trigger Enter logic from earlier lessons) and add an Update event node - it fires once per frame, the same as Update() in the C# path.
5. Feed the Update event into a "Translate" node (Space: World), with its Translation input set to Vector3.down * enemySpeed * deltaTime - the same expression as transform.Translate(Vector3.down * enemySpeed * Time.deltaTime) in the code path.
6. From the same Update event, add a Branch node whose Condition compares the Enemy's transform.position.y against your off-screen threshold (transform.position.y < -6f), with its True output feeding a Destroy node targeting itself.
7. Confirm the Destroy node here is a plain destroy, with no call into the GameManager's kill-registering logic - that stays exclusive to the Bullet's own graph, exactly as in the C# path.

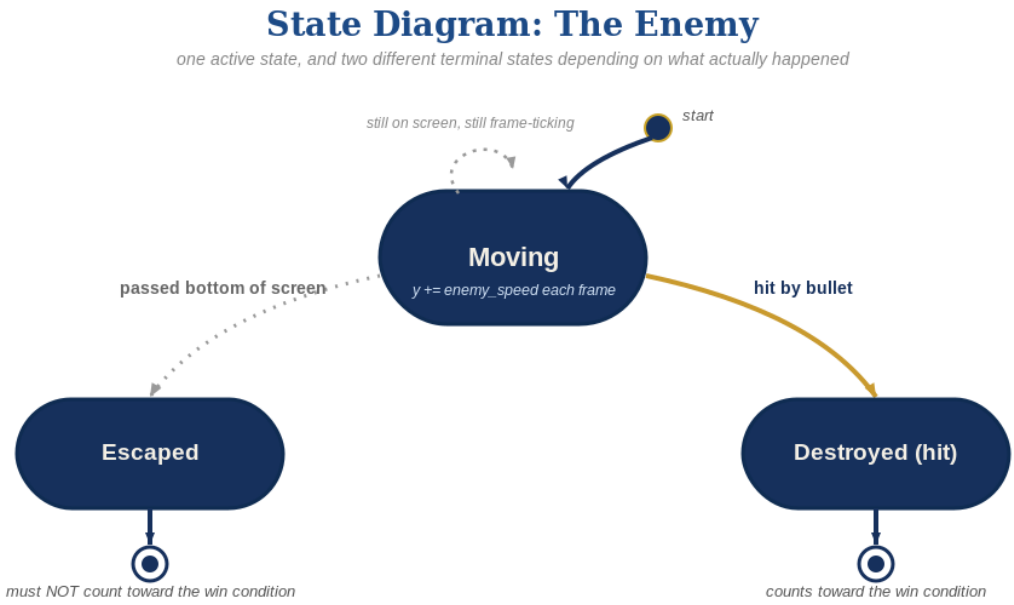


The Enemy prefab's Update graph, in Unity Visual Scripting - the node-for-node equivalent of the Level 1 code.

Level 2/3 already work the same way as the C# path, since the escape-vs-hit distinction lives in which graph calls Destroy, not in any engine-specific detail.

Level 2 - Complete (Alpha/Beta)

A moving enemy can still be shot and destroyed correctly, and an enemy that escapes off the bottom never triggers "You Win!" by accident.



Once an enemy leaves Moving, it never comes back - both destinations are final. The whole trick this lesson teaches is making sure your code can tell these two final states apart, since only one of them is a kill.

The enemy's two possible endings this week - only one of them should count as a kill.

- Shoot a moving enemy to confirm Bullet.cs's OnTriggerEnter2D still fires correctly while it's in motion.
- Let at least one enemy escape off the bottom unshot, and confirm "You Win!" does NOT appear - only Bullet.cs's OnTriggerEnter2D calls GameManager.RegisterKill(), and Enemy.cs's own Update() never calls it.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Visual: give each enemy a slightly randomized enemySpeed in Start() (e.g. enemySpeed = Random.Range(1f, 2f)) so they don't all move in perfect lockstep.
- Structure: keep Enemy.cs's own Update() cleanup as a plain Destroy(gameObject) only - never let it call GameManager.RegisterKill(), which is reserved exclusively for an actual hit in Bullet.cs.

If Something's Wrong

- Enemies don't move at all: confirm Update() is spelled exactly right (case-sensitive) and enemySpeed is actually greater than 0.
- "You Win!" appears even when enemies escaped unshot: confirm Enemy.cs's Update() never calls GameManager.RegisterKill() - that call belongs exclusively in Bullet.cs.
- Enemies move but never disappear at the bottom: confirm the transform.position.y < -6f threshold actually matches your camera's visible area - adjust the number if your camera setup differs.

Want More?

Student Resource List > Unity section, or learn.unity.com